Milestone 3: Data Population/SQL Queries
Stock News Trader

Philip Lee, Kartheek Gavini, Na Ni

SQL Queries

Query 1 (Simple/Moderate) — Company Search By ticker or name

Description: This query enables users to search companies by ticker symbol or company name and additionally returns their corresponding industry and sector information, along with the number of companies within the same industry. It filters results to include only companies belonging to industries that have above-average company counts, using joins, aggregation, subqueries, and analytical window functions.

```
-- Param: $1 = ticker
-- Param: $2 = name
SELECT
   c.company_id,
   c.ticker,
   c.company_name,
   i.industry_name,
   s.sector_name,
   COUNT(*) OVER (PARTITION BY i.industry_id) AS companies_in_industry
FROM company c
JOIN public."Industry" i
   ON c.industry_id = i.industry_id
JOIN public."Sector" s
   ON i.sector_id = s.sector_id
WHERE (
      c.ticker ILIKE ($1 || '%')
    OR c.company_name ILIKE ($2 || '%')
)
AND c.industry_id IN (
   SELECT industry_id
   FROM company
   GROUP BY industry_id
```

```
    HAVING COUNT(*) > (
        SELECT AVG(industry_count)
        FROM (
            SELECT COUNT(*) AS industry_count
            FROM company
            GROUP BY industry_id
        ) industry_avg
    )
)
ORDER BY companies_in_industry DESC, c.ticker;
```

Query 2 (Simple/Moderate) — S&P 500 Company Information Detail

Description: This query retrieves S&P 500 companies together with their industry and sector information and additionally identifies companies that belong to industries with above-average numbers of companies within their sectors. It uses multiple joins, aggregation, nested subqueries, window functions, and filtering conditions to analyze the distribution of companies across industries and sectors.

```
SELECT
    c.company_id,
    c.ticker,
    c.company_name,
    i.industry_name,
    s.sector_name,
    COUNT(c.company_id) OVER (PARTITION BY i.industry_id) AS
companies_in_industry,
    COUNT(c.company_id) OVER (PARTITION BY s.sector_id) AS companies_in_sector
FROM company c
JOIN public."Industry" i
    ON c.industry_id = i.industry_id
JOIN public."Sector" s
    ON i.sector_id = s.sector_id
WHERE i.industry_id IN (
    SELECT industry_id
    FROM company
    GROUP BY industry_id
    HAVING COUNT(*) > (
        SELECT AVG(industry_count)
```

```
      FROM (
        SELECT COUNT(*) AS industry_count
        FROM company
        GROUP BY industry_id
      ) avg_industry_counts
    )
)
ORDER BY companies_in_sector DESC,
       companies_in_industry DESC,
       c.ticker;
```

Query 3 (Complex) — TOP 10 Gainers in one day

Description: This query identifies the top 10 daily stock gainers for a selected trading date by comparing closing prices with the most recent prior trading day. It joins company classification tables to include sector and industry information, calculates sector-level average returns, ranks companies within their sectors, and filters results to include only companies outperforming their sector averages. The query uses multiple joins, CTEs, nested subqueries, aggregation, window functions, and ranking logic.

```
-- Param: $1 = date
WITH latest AS (
  SELECT
    company_id,
    close,
    trading_date
  FROM price_daily
  WHERE trading_date = $1
),

prev AS (
  SELECT
    company_id,
    close AS prev_close
  FROM price_daily
  WHERE trading_date = (
    SELECT MAX(trading_date)
    FROM price_daily
    WHERE trading_date < $1
```

```
    )
),

price_change AS (
    SELECT
        l.company_id,
        l.close,
        p.prev_close,
        ((l.close - p.prev_close) / p.prev_close) * 100 AS pct_change
    FROM latest l
    JOIN prev p
        ON l.company_id = p.company_id
),

sector_avg AS (
    SELECT
        i.sector_id,
        AVG(((l.close - p.prev_close) / p.prev_close) * 100) AS avg_sector_change
    FROM latest l
    JOIN prev p
        ON l.company_id = p.company_id
    JOIN company c
        ON c.company_id = l.company_id
    JOIN public."Industry" i
        ON c.industry_id = i.industry_id
    GROUP BY i.sector_id
)

SELECT
    c.ticker,
    c.company_name,
    s.sector_name,
    i.industry_name,
    pc.close,
    pc.prev_close,
    pc.pct_change,
    sa.avg_sector_change,
    RANK() OVER (PARTITION BY s.sector_id ORDER BY pc.pct_change DESC) AS
sector_rank
FROM price_change pc
```

```
JOIN company c
   ON pc.company_id = c.company_id
JOIN public."Industry" i
   ON c.industry_id = i.industry_id
JOIN public."Sector" s
   ON i.sector_id = s.sector_id
JOIN sector_avg sa
   ON sa.sector_id = s.sector_id
WHERE pc.pct_change > sa.avg_sector_change
ORDER BY pc.pct_change DESC
LIMIT 10;
```

Query 4 (Complex) — Historical closing price for selected company

Description: This query retrieves historical daily prices for a selected company and enriches the time-series output with daily return percentages, 7-day and 30-day moving averages, sector average closing prices, and price ranking within the sector on each trading date. It uses multiple joins, window functions, aggregations, and subqueries to support advanced time-series analytics suitable for financial charting and benchmarking.

```
-- Params: $1 = company_id (int)

WITH sector_avg AS (
   SELECT
      pd.trading_date,
      i.sector_id,
      AVG(pd.close) AS sector_avg_close
   FROM price_daily pd
   JOIN company c
      ON pd.company_id = c.company_id
   JOIN public."Industry" i
      ON c.industry_id = i.industry_id
   GROUP BY pd.trading_date, i.sector_id
)

SELECT
   c.company_name,
   c.ticker,
```

```
    s.sector_name,
    i.industry_name,
    pd.trading_date,
    pd.close,

    -- Daily return %
    ((pd.close - LAG(pd.close)
      OVER (PARTITION BY pd.company_id ORDER BY pd.trading_date))
      / LAG(pd.close)
      OVER (PARTITION BY pd.company_id ORDER BY pd.trading_date)) * 100
      AS daily_return_pct,

    -- 7-day moving average
    AVG(pd.close)
      OVER (PARTITION BY pd.company_id
            ORDER BY pd.trading_date
            ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)
      AS ma_7_day,

    -- 30-day moving average
    AVG(pd.close)
      OVER (PARTITION BY pd.company_id
            ORDER BY pd.trading_date
            ROWS BETWEEN 29 PRECEDING AND CURRENT ROW)
      AS ma_30_day,

    sa.sector_avg_close,

    -- Rank within sector by price each day
    RANK() OVER (
      PARTITION BY s.sector_name, pd.trading_date
      ORDER BY pd.close DESC
    ) AS sector_price_rank

FROM price_daily pd
JOIN company c
  ON pd.company_id = c.company_id
JOIN public."Industry" i
  ON c.industry_id = i.industry_id
JOIN public."Sector" s
```

```
   ON i.sector_id = s.sector_id
JOIN sector_avg sa
   ON sa.trading_date = pd.trading_date
   AND sa.sector_id = s.sector_id

WHERE pd.company_id = $1

ORDER BY pd.trading_date;
```

Query 5 (Complex): Top 5 Companies with Highest Average Daily Return in the Last Month

Description: This query calculates the average daily return for each company over the last 30 days, ranks companies by average return, and filters to return only the top 5 performing companies. It also shows sector and industry for comparison and calculates volatility (standard deviation of returns) to help identify high-risk, high-return companies.

```
WITH daily_returns AS (
   SELECT
      pd.company_id,
      pd.trading_date,
      (pd.close - LAG(pd.close) OVER (PARTITION BY pd.company_id ORDER BY
pd.trading_date)) /
      LAG(pd.close) OVER (PARTITION BY pd.company_id ORDER BY
pd.trading_date) AS daily_return
   FROM price_daily pd
   WHERE pd.trading_date >= CURRENT_DATE - INTERVAL '30 days'
),

return_stats AS (
   SELECT
      dr.company_id,
      AVG(dr.daily_return) AS avg_daily_return,
      STDDEV(dr.daily_return) AS return_volatility
   FROM daily_returns dr
   GROUP BY dr.company_id
)

SELECT
```

```
    c.ticker,
    c.company_name,
    i.industry_name,
    s.sector_name,
    rs.avg_daily_return,
    rs.return_volatility,
    RANK() OVER (ORDER BY rs.avg_daily_return DESC) AS return_rank
FROM return_stats rs
JOIN company c ON rs.company_id = c.company_id
JOIN public."Industry" i ON c.industry_id = i.industry_id
JOIN public."Sector" s ON i.sector_id = s.sector_id
ORDER BY return_rank
LIMIT 5;
```

Query 6: Sector-wise Momentum Analysis – Companies Outperforming Their Sector in Last 7 Days

Description**:** This query identifies companies whose 7-day cumulative returns exceed the average cumulative return of their sector over the same period. It ranks companies within each sector, allowing for sector-relative momentum analysis.

```
WITH recent_prices AS (
    SELECT
        pd.company_id,
        pd.trading_date,
        pd.close,
        LAG(pd.close, 7) OVER (PARTITION BY pd.company_id ORDER BY pd.trading_date) AS
close_7_days_ago
    FROM price_daily pd
    WHERE pd.trading_date >= CURRENT_DATE - INTERVAL '7 days'
),

cumulative_returns AS (
    SELECT
        rp.company_id,
        ((rp.close - rp.close_7_days_ago) / rp.close_7_days_ago) * 100 AS return_7_days
    FROM recent_prices rp
    WHERE rp.close_7_days_ago IS NOT NULL
),

sector_avg_returns AS (
    SELECT
```

```
        i.sector_id,
        AVG(cr.return_7_days) AS avg_sector_return
    FROM cumulative_returns cr
    JOIN company c ON cr.company_id = c.company_id
    JOIN public."Industry" i ON c.industry_id = i.industry_id
    GROUP BY i.sector_id
)

SELECT
    c.ticker,
    c.company_name,
    i.industry_name,
    s.sector_name,
    cr.return_7_days,
    sar.avg_sector_return,
    RANK() OVER (PARTITION BY s.sector_id ORDER BY cr.return_7_days DESC) AS
sector_rank
FROM cumulative_returns cr
JOIN company c ON cr.company_id = c.company_id
JOIN public."Industry" i ON c.industry_id = i.industry_id
JOIN public."Sector" s ON i.sector_id = s.sector_id
JOIN sector_avg_returns sar ON sar.sector_id = s.sector_id
WHERE cr.return_7_days > sar.avg_sector_return
ORDER BY s.sector_name, sector_rank;
```

Query 7 (Complex) – Top 10 companies mention most in the news

Description: This query identifies the top 10 most-mentioned companies in recent news articles (last 30 days) and enriches the results with their sector and industry classifications, sector-level average mention counts, and rankings within each sector. It filters companies whose mention counts exceed their sector's average, using multiple joins, aggregations, nested subqueries, window functions, and ranking logic.

```
WITH mention_counts AS (
    SELECT
        am.company_id,
        COUNT(*) AS mention_count
    FROM article_mention am
    JOIN news_article na
        ON am.article_id = na.article_id
    WHERE na.published_at >= CURRENT_DATE - INTERVAL '30 days'
    GROUP BY am.company_id
```

```
),

sector_avg_mentions AS (
    SELECT
        i.sector_id,
        AVG(mc.mention_count) AS avg_sector_mentions
    FROM mention_counts mc
    JOIN company c
        ON mc.company_id = c.company_id
    JOIN public."Industry" i
        ON c.industry_id = i.industry_id
    GROUP BY i.sector_id
)

SELECT
    c.ticker,
    c.company_name,
    s.sector_name,
    i.industry_name,
    mc.mention_count,
    sam.avg_sector_mentions,

    RANK() OVER (
        PARTITION BY s.sector_id
        ORDER BY mc.mention_count DESC
    ) AS sector_rank

FROM mention_counts mc
JOIN company c
    ON mc.company_id = c.company_id
JOIN public."Industry" i
    ON c.industry_id = i.industry_id
JOIN public."Sector" s
    ON i.sector_id = s.sector_id
JOIN sector_avg_mentions sam
    ON sam.sector_id = s.sector_id

WHERE mc.mention_count > sam.avg_sector_mentions

AND EXISTS (
    SELECT 1
    FROM article_mention am2
    JOIN news_article na2
        ON am2.article_id = na2.article_id
```

```
    WHERE am2.company_id = mc.company_id
      AND na2.published_at >= CURRENT_DATE - INTERVAL '30 days'
    GROUP BY am2.company_id
    HAVING COUNT(DISTINCT na2.article_id) >= 5
)

ORDER BY mc.mention_count DESC
LIMIT 10;
```

Query 8 (Complex): Top 10 most recent news for selected company

Description**:** This query retrieves the 10 most recent news articles mentioning a selected company, enriched with the company's sector and industry information, article mention frequency within the last 30 days, and ranking by recency and relevance. It filters articles to include only those published within the recent time window and whose mention frequency exceeds the company's average mention rate. The query uses multiple joins, CTEs, aggregations, nested subqueries, window functions, and ranking logic to support advanced news analytics.

```
--param: $1 = company_id(int);

WITH recent_mentions AS (
    SELECT
        am.article_id,
        am.company_id,
        COUNT(*) AS mention_frequency
    FROM article_mention am
    JOIN news_article na
        ON am.article_id = na.article_id
    WHERE na.published_at >= CURRENT_DATE - INTERVAL '30 days'
    GROUP BY am.article_id, am.company_id
),

company_avg_mentions AS (
    SELECT
        company_id,
        AVG(mention_frequency) AS avg_mentions
    FROM recent_mentions
    GROUP BY company_id
```

```
)

SELECT
   c.company_name,
   c.ticker,
   s.sector_name,
   i.industry_name,
   na.title,
   na.summary,
   na.published_at,
   na.url,
   rm.mention_frequency,

   RANK() OVER (
      PARTITION BY c.company_id
      ORDER BY na.published_at DESC
   ) AS recency_rank

FROM news_article na
JOIN article_mention am
   ON na.article_id = am.article_id
JOIN company c
   ON am.company_id = c.company_id
JOIN public."Industry" i
   ON c.industry_id = i.industry_id
JOIN public."Sector" s
   ON i.sector_id = s.sector_id
JOIN recent_mentions rm
   ON rm.article_id = na.article_id
  AND rm.company_id = c.company_id
JOIN company_avg_mentions cam
   ON cam.company_id = c.company_id

WHERE c.company_id = $1
AND rm.mention_frequency >= cam.avg_mentions

AND EXISTS (
   SELECT 1
   FROM news_article na2
   WHERE na2.article_id = na.article_id
```

```
    AND na2.published_at >= CURRENT_DATE - INTERVAL '30 days'
)

ORDER BY na.published_at DESC
LIMIT 10;
```

Query 9 (Complex) — Multi-source disagreement: biggest cross-source close spread - Philip

Description**:** Quantify disagreement across multiple price sources by computing per-day close spread (max–min) and reporting the largest spreads per ticker (aggregation + window ranking).

```
-- Params: $1 = start_date, $2 = end_date, $3 = min_sources (INT)
WITH spans AS (
  SELECT
    pd.company_id,
    pd.trading_date,
    COUNT(*) AS n_sources,
    MAX(pd.close) - MIN(pd.close) AS close_spread
  FROM price_daily pd
  WHERE pd.trading_date BETWEEN $1 AND $2
    AND pd.close IS NOT NULL
  GROUP BY pd.company_id, pd.trading_date
),
ranked AS (
  SELECT
    c.ticker,
    s.trading_date,
    s.n_sources,
    s.close_spread,
    ROW_NUMBER() OVER (PARTITION BY c.ticker ORDER BY s.close_spread DESC)
AS rn
  FROM spans s
  JOIN company c ON c.company_id = s.company_id
  WHERE s.n_sources >= $3
)
SELECT *
FROM ranked
WHERE rn = 1
```

```
ORDER BY close_spread DESC
LIMIT 50;
```

Query 10 (Complex) — Biggest month-over-month rank changes for industries (within sector) - Philip

Description: Track monthly industry performance and compute the biggest month-over-month rank changes within each sector to identify "rotations" (CTEs + windows + multi-level aggregation).

```
-- Params: $1 = start_date, $2 = end_date
WITH primary_prices AS (
  SELECT DISTINCT ON (pd.company_id, pd.trading_date)
    pd.company_id, pd.trading_date, pd.close
  FROM price_daily pd
  JOIN price_source ps ON ps.source_id = pd.source_id
  WHERE pd.trading_date BETWEEN $1 AND $2
  ORDER BY pd.company_id, pd.trading_date, ps.priority_rank ASC
),
rets AS (
  SELECT
    company_id,
    trading_date,
    (close / NULLIF(LAG(close) OVER (PARTITION BY company_id ORDER BY trading_date),
0) - 1) AS ret
  FROM primary_prices
),
company_month AS (
  SELECT
    c.company_id,
    date_trunc('month', r.trading_date)::date AS month,
    EXP(SUM(LN(1 + r.ret))) - 1 AS month_ret
  FROM rets r
  JOIN company c ON c.company_id = r.company_id
  WHERE r.ret IS NOT NULL
  GROUP BY c.company_id, date_trunc('month', r.trading_date)::date
),
industry_month AS (
  SELECT
    s.sector_name,
```

```
    i.industry_name,
    cm.month,
    AVG(cm.month_ret) AS industry_month_ret
  FROM company_month cm
  JOIN company c  ON c.company_id = cm.company_id
  JOIN public."Industry" i ON i.industry_id = c.industry_id
  JOIN public."Sector" s   ON s.sector_id = i.sector_id
  GROUP BY s.sector_name, i.industry_name, cm.month
),
ranked AS (
  SELECT
    *,
    RANK() OVER (PARTITION BY sector_name, month ORDER BY industry_month_ret DESC)
AS rnk
  FROM industry_month
),
deltas AS (
  SELECT
    sector_name, industry_name, month, industry_month_ret, rnk,
    LAG(rnk) OVER (PARTITION BY sector_name, industry_name ORDER BY month) AS
prev_rnk
  FROM ranked
)
SELECT
  sector_name, industry_name, month,
  prev_rnk, rnk,
  (prev_rnk - rnk) AS rank_improvement,
  industry_month_ret
FROM deltas
WHERE prev_rnk IS NOT NULL
ORDER BY ABS(prev_rnk - rnk) DESC
LIMIT 50;
```

Credentials

Host: project.ct8q4usyak98.us-east-1.rds.amazonaws.com
Port: 5432
UserName: student
password: cis5500$
Database: Project